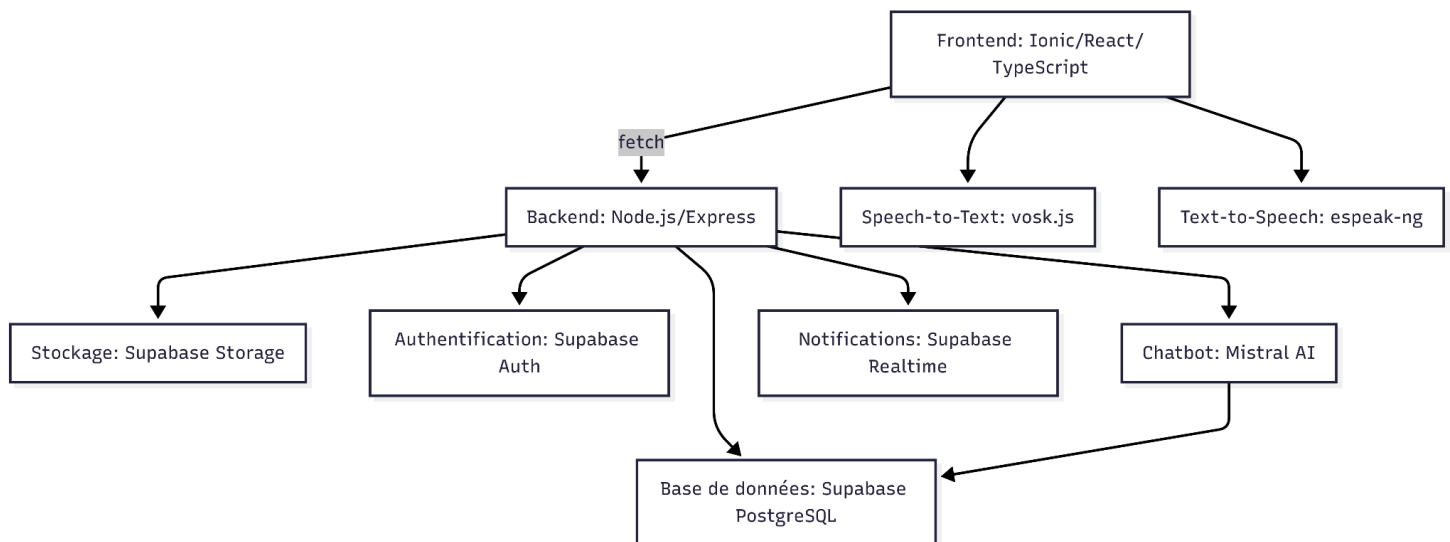


Projet B - Séquence 3.5 - Architecture technique et moyens

Merci pour ces précisions ! Voici une **architecture technique finalisée** et une **plateforme de développement** adaptées à tes choix, avec des exemples concrets pour chaque composant. Je vais aussi identifier les **risques** et **conditions de succès**, ainsi que les **tâches à réaliser** pour chaque partie.

Architecture Technique Finalisée

Schéma global



Détail par Composant

1 Frontend (Ionic + React + TypeScript)

Stack

- **Framework** : Ionic CLI + **React** + **TypeScript** (pour une meilleure typage et maintenabilité).
- **UI** :
 - Slide-menu (accueil, compte, déconnexion).
 - Page principale avec image animée synchronisée à la voix + bouton on/off.
 - Pages : transcriptions, identification/inscription, calendrier, groupes collaboratifs.
- **Bibliothèques clés** :

- **Speech-to-Text** : [vosk.js](#) (offline, modèles légers).
 - Exemple d'intégration :

 typescript

```
import { Model, KaldiRecognizer } from 'vosk';
const model = await Model.load('path/to/model');
const recognizer = new KaldiRecognizer(model, 16000);
recognizer.on('result', (result) => console.log(result));
```

- **Text-to-Speech** : [espeak-ng](#) (via un **backend Node.js** pour générer l'audio et le renvoyer au frontend).
 - **Pourquoi pas côté frontend ?** : [espeak-ng](#) est un binaire système, donc plus simple à appeler depuis le backend.
 - Alternative : Utiliser l'[API Web Speech](#) (online) si [espeak-ng](#) pose problème.
- **Appels REST** : [fetch\(\)](#) (natif, pas besoin de bibliothèque supplémentaire).
- **Supabase Client** : [@supabase/supabase-js](#) pour l'authentification et le stockage.

Exemple de structure de dossiers

text  Copier

```
src/
├── components/
│   ├── VoiceButton.tsx    # Bouton on/off pour la voix
│   ├── Calendar.tsx      # Calendrier des tâches
│   └── Chatbot.tsx        # Interface du chatbot
├── pages/
│   ├── Home.tsx          # Page principale
│   ├── Transcriptions.tsx # Liste des transcriptions
│   ├── Login.tsx         # Connexion
│   └── Groups.tsx         # Groupes collaboratifs
├── services/
│   ├── api.ts             # Appels REST vers le backend
│   ├── supabase.ts        # Client Supabase
│   └── voice.ts            # Logique pour vosk.js/espeak-ng
└── App.tsx
```

Exemple de code pour **voice.ts** (Speech-to-Text)

Exemple de code pour `voice.ts` (Speech-to-Text)

TS typescript

Copier

```
import { Model, KaldiRecognizer } from 'vosk';

let recognizer: KaldiRecognizer | null = null;
let model: Model | null = null;

export const initVoiceRecognition = async (modelPath: string) => {
  model = await Model.load(modelPath);
  recognizer = new KaldiRecognizer(model, 16000);
  recognizer.on('result', (result) => {
    console.log('Transcription:', result);
    // Envoyer la transcription au backend ou au chatbot
  });
};

export const startListening = (stream: MediaStream) => {
  if (!recognizer) throw new Error('Recognizer not initialized');
  const audioContext = new AudioContext();
  const source = audioContext.createMediaStreamSource(stream);
  const processor = audioContext.createScriptProcessor(4096, 1, 1);
  source.connect(processor);
  processor.connect(audioContext.destination);
  processor.onaudioprocess = (e) => {
    const data = e.inputBuffer.getChannelData(0);
    recognizer?.acceptWaveform(data);
  };
};
```

2 Backend (Node.js + Express)

Stack

- **Framework** : Express.js (léger et flexible).
- **Bibliothèques clés** :
 - **Supabase** : [@supabase/supabase-js](#) (pour l'authentification et le stockage).
 - **Mistral AI** : [axios](#) pour les appels à l'API.
 - **Text-to-Speech** : Appel à [espeak-ng](#) via [child_process](#).
 - **Notifications** : Supabase Realtime (pour notifier les utilisateurs en temps réel).
 - **Calendrier/Groupes** : Logique métier pour créer des tâches, groupes, et notifications.

Exemple de structure de dossiers

text

 Copier

```
backend/  
├── src/  
│   ├── controllers/  
│   │   ├── auth.ts           # Authentification Supabase  
│   │   ├── tasks.ts         # Gestion des tâches  
│   │   ├── groups.ts        # Gestion des groupes  
│   │   └── chatbot.ts        # Logique du chatbot  
│   ├── routes/  
│   │   ├── auth.ts  
│   │   ├── tasks.ts  
│   │   ├── groups.ts  
│   │   └── chatbot.ts  
│   ├── services/  
│   │   ├── supabase.ts      # Client Supabase  
│   │   ├── mistral.ts       # Appels à Mistral AI  
│   │   ├── tts.ts           # Text-to-Speech avec espeak-ng  
│   └── app.ts               # Configuration Express  
├── .env  
├── package.json  
└── Dockerfile
```

Exemple de code pour **services/tts.ts** (Text-to-Speech)

 typescript

 Copier

```
import { exec } from 'child_process';  
import { promisify } from 'util';  
const execAsync = promisify(exec);  
  
export const textToSpeech = async (text: string): Promise<Buffer> => {  
    const { stdout } = await execAsync(`espeak-ng -v fr -s 150 "${text}" --stdout`);  
    return Buffer.from(stdout, 'binary');  
};
```

Exemple de code pour **services/mistral.ts** (Chatbot)

TS typescript

 Copier

```
import axios from 'axios';

const MISTRAL_API_KEY = process.env.MISTRAL_API_KEY;

export const askMistral = async (prompt: string, userId: string) => {
  // 1. Récupérer les données de l'utilisateur depuis Supabase
  const userData = await getUserData(userId); // À implémenter
  const context = `Données de l'utilisateur: ${JSON.stringify(userData)}. Question: ${prompt}`;

  // 2. Appeler Mistral AI
  const response = await axios.post(
    'https://api.mistral.ai/v1/chat/completions',
    {
      model: 'mistral-medium',
      messages: [{ role: 'user', content: context }],
    },
    { headers: { Authorization: `Bearer ${MISTRAL_API_KEY}` } }
  );

  const answer = response.data.choices[0].message.content;

  // 3. Analyser la réponse pour déclencher des actions (ex: créer une tâche)
  if (answer.includes('Créer une tâche')) {
    await createTaskFromAnswer(answer, userId); // À implémenter
  } else if (answer.includes('Créer un groupe')) {
    await createGroupFromAnswer(answer, userId); // À implémenter
  } else if (answer.includes('Envoyer une notification')) {
    await sendNotificationFromAnswer(answer, userId); // À implémenter
  }

  return answer;
};
```

Exemple de code pour [controllers/chatbot.ts](#)

ts typescript

Copier

```
import { Request, Response } from 'express';
import { askMistral } from '../services/mistral';

export const chat = async (req: Request, res: Response) => {
  const { prompt, userId } = req.body;
  try {
    const answer = await askMistral(prompt, userId);
    res.json({ answer });
  } catch (error) {
    res.status(500).json({ error: 'Erreur avec Mistral AI' });
  }
};
```

3 Base de Données (Supabase PostgreSQL)

Tables principales

Table	Champs	Description
users	id (uuid), email, password_hash, name	Utilisateurs (gérés par Supabase Auth).
tasks	id (uuid), user_id (uuid), title, description, due_date, calendar_event_id	Tâches de l'utilisateur.
groups	id (uuid), name, creator_id (uuid)	Groupes collaboratifs.
group_members	group_id (uuid), user_id (uuid)	Membres des groupes.
transcriptions	id (uuid), user_id (uuid), content, timestamp	Transcriptions vocales.
notifications	id (uuid), user_id (uuid), message, is_read, created_at	Notifications pour les utilisateurs.
chatbot_sessions	id (uuid), user_id (uuid), messages (jsonb)	Historique des conversations avec le chatbot.

Recherche full-text

- Utiliser `tsvector` pour les champs `content` (transcriptions) et `title/description` (tâches).
- Exemple de requête :

```
sql

SELECT * FROM transcriptions
WHERE to_tsvector('french', content) @@ to_tsquery('french', 'recherche & texte');
```

Supabase Realtime

- Pour les **notifications en temps réel** :

```
js javascript

// Frontend
supabase
  .channel('notifications')
  .on('postgres_changes', { event: 'INSERT', schema: 'public', table:
'notifications' }, (payload) => {
    console.log('Nouvelle notification:', payload.new);
  })
  .subscribe();
```

4 Authentification (Supabase Auth)

- **Intégration** :
 - Utiliser [@supabase/supabase-js](https://supabase.com/docs/reference/javascript) pour :
 - Inscription (`signUp`).
 - Connexion (`signIn`).
 - Déconnexion (`signOut`).
 - **Exemple de code frontend** :

```
ts typescript

import { createClient } from '@supabase/supabase-js';
const supabase = createClient(process.env.SUPABASE_URL!,
process.env.SUPABASE_KEY!);

// Connexion
const { data, error } = await supabase.auth.signInWithPassword({
  email: 'user@example.com',
  password: 'password',
});
```

Plateforme de Développement

1 Éditeur (VSCode)

Extensions recommandées

Extension	Lien	Utilité
Ionic	Lien	Développement Ionic.
ESLint	Lien	Linting TypeScript.
Prettier	Lien	Formatage de code.
Docker	Lien	Gestion des conteneurs.
Supabase Helper	Lien	Intégration Supabase.
Vibe Coding	Lien	Intégration Vibe/Mistral.
GitHub Pull Requests	Lien	Gestion des PR GitHub.

2 GitHub

Structure du dépôt

GitHub Projects

- **Tableau Kanban** pour suivre :
 - **Backlog** : Fonctionnalités à développer.
 - **In Progress** : Tâches en cours.
 - **Review** : PR à revoir.
 - **Done** : Tâches terminées.

GitHub Actions (CI/CD)

- **Exemple de workflow** (`/.github/workflows/deploy.yml`) :

```
YAML yaml Copier
name: Deploy Backend
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t studyflow-backend ./backend
      - name: Deploy to VM
        run: |
          echo "${{ secrets.SSH_PRIVATE_KEY }}" > key.pem
          chmod 400 key.pem
          scp -i key.pem -o StrictHostKeyChecking=no docker-compose.yml user@ton-
serveur:/home/user/
          ssh -i key.pem -o StrictHostKeyChecking=no user@ton-serveur "docker-
compose down && docker-compose up -d"
```

- **Secrets GitHub** à configurer :
 - **SSH_PRIVATE_KEY** : Clé SSH pour accéder à ta VM Linux.

⚠ Risques et Conditions de Succès

● Risques

Risque	Impact	Mitigation
Latence de Mistral AI	Réponses lentes du chatbot.	Utiliser un cache (Redis) pour les requêtes fréquentes.
Modèles vosk.js lourds	Performance faible sur mobile.	Choisir un modèle léger (ex : <code>vosk-model-small-fr-0.22</code>).
espeak-ng non disponible	Text-to-Speech ne fonctionne pas.	Pré-installer <code>espeak-ng</code> sur la VM et dans le conteneur Docker.
Sécurité des JWT	Fuites de tokens.	Utiliser <code>httpOnly</code> et <code>secure</code> pour les cookies, et limiter leur durée de vie.

Supabase Realtime instable	Notifications non reçues.	Implémenter un fallback (polling).
Déploiement manuel sur VM	Erreurs humaines.	Automatiser avec GitHub Actions + scripts de déploiement.

● Conditions de Succès

Condition	Action
Modèles vosk.js téléchargés	Télécharger les modèles français et les inclure dans le projet.
Clé API Mistral valide	Obtenir une clé API et la stocker dans <code>.env</code> .
Supabase configuré	Créer un projet Supabase, activer Auth et Storage, et configurer les RLS.
VM Linux prête	Installer Docker, <code>espeak-ng</code> , et ouvrir les ports nécessaires (3000, 5432).
Tests unitaires	Écrire des tests pour le backend (Jest) et le frontend (React Testing Library).
Monitoring	Mettre en place des logs (ex : <code>winston</code> pour Node.js) et un outil comme Prometheus/Grafana.

Tâches à Réaliser (par ordre de priorité)

◆ Phase 1 : Setup Initial (1-2 jours)

- Créer le projet Supabase (base de données + auth + storage).
- Configurer les tables (users, tasks, groups, etc.) et les RLS.
- Initialiser le dépôt GitHub avec la structure de dossiers.
- Configurer VSCode avec les extensions nécessaires.
- Installer `espeak-ng` sur ta VM Linux :

◆ Phase 2 : Backend (3-5 jours)

- Créer le projet Node.js/Express avec TypeScript.
- Intégrer Supabase Auth (`@supabase/supabase-js`).
- Implémenter les routes REST :
 - Authentification (`/auth`).
 - Tâches (`/tasks`).

- Groupes (/groups).
- Chatbot (/chatbot).
- Ajouter la logique pour **espeak-ng** (Text-to-Speech).
- Intégrer **Mistral AI** (appels API + traitement des réponses).
- Configurer **Supabase Realtime** pour les notifications.
- Écrire des tests unitaires (Jest).

◆ Phase 3 : Frontend (5-7 jours)

- Créer le projet **Ionic/React/TypeScript**.
- Intégrer **vosk.js** pour le Speech-to-Text.
- Créer les pages :
 - Accueil (avec image animée + bouton voix).
 - Connexion/Inscription.
 - Transcriptions.
 - Calendrier (affichage des tâches).
 - Groupes collaboratifs.
 - Chatbot.
- Appeler le backend via **fetch()**.
- Gérer les notifications avec Supabase Realtime.
- Tester sur mobile (Ionic DevApp ou build APK/IPA).

◆ Phase 4 : Docker et Déploiement (2-3 jours)

- Créer le **Dockerfile** pour le backend.
- Configurer **docker-compose.yml** pour le développement local.
- Automatiser le déploiement avec GitHub Actions.
- Déployer sur la VM Linux et tester.

◆ Phase 5 : Tests et Optimisations (2-3 jours)

- Tester toutes les fonctionnalités (auth, chatbot, voix, etc.).
- Optimiser les performances (ex : cache pour Mistral AI).
- Corriger les bugs identifiés.
- Documenter le projet (README, commentaires dans le code).

Prochaines Étapes

1. Valider les choix :

- Confirmes-tu que cette architecture te convient ?
- Veux-tu que je te fournisse des **exemples de code complets** pour un composant spécifique (ex : backend Node.js, frontend Ionic) ?

2. Commencer par où ? :

- Veux-tu que je te génère un **template de projet** (structure de dossiers + fichiers de base) pour le frontend et/ou le backend ?
- Ou préfères-tu que je détaille davantage un composant (ex : intégration de `vosk.js`, appels à Mistral AI) ?

3. Outils supplémentaires :

- Veux-tu que je te propose un **schéma d'architecture visuel** (Mermaid ou SVG) ?
 - Ou un **canvas** pour collaborer sur le code (ex : backend Node.js) ?
-

Exemple de réponse attendue :

"Je valide l'architecture. Peux-tu me générer un template de projet pour le backend Node.js avec les fichiers de base (app.ts, routes, services) ? Et aussi un exemple de Dockerfile pour le backend."

À toi de jouer ! 🚀